

Review Cheat Sheet — A Readable Companion to the Elixir Docs

Chapter 14 · all checklists in one place · Sep 26, 2026

How to use this sheet

This is every chapter’s checklist, regrouped by the question a reviewer asks. The number after each item is the chapter that explains it. Start with the red flags to find suspicious code quickly, then use the sections to review it properly. For runtime topics (memory, ETS, distribution, schedulers), use the cheat sheet in the companion *Learning Erlang from Claude* as well.

1. Red flags you can search for

These patterns are not always wrong, but each one deserves a second look.

Search for	Why it’s suspicious	Ch.
<code>map[:field]</code> on data with required fields	Missing keys become <code>nil</code> far from the cause	1
<code>or</code> combining type checks in a guard	One failing check fails the whole guard	1
<code>if System.get_env(...)</code>	<code>"false"</code> is truthy	1
<code>defaults ++ opts</code>	The first duplicate key wins; caller options ignored	1
<code>struct(Mod, params)</code>	Ignores <code>@enforce_keys</code>	1
<code>rescue _ -></code> or <code>rescue</code> around parsing	Hides bugs; exceptions as control flow	2
<code>true <-</code> or <code>false <-</code> in <code>with</code>	Leaks a boolean instead of an error tuple	2
<code>Task.async</code> inside <code>handle_*</code> callbacks	Linked crash plus blocked server	3
<code>Task.async_stream(...)</code> without <code>Stream.run/Enum</code>	Never runs	3
<code>Agent.get</code> followed by <code>Agent.update</code>	Lost updates	3
<code>Task.start()</code> outside a supervisor	Unsupervised process	3
<code>name: __MODULE__</code> hard-coded	Single instance; test collisions	4, 11

Search for	Why it's suspicious	Ch.
<code>GenServer.call(__MODULE__</code> called from a callback	Exits with <code>:calling_self</code>	4
<code>def handle_info</code> without a catch-all clause	Crashes on stray messages	4
<code>DynamicSupervisor</code> with <code>:permanent</code> children	A few crashes kill every child	5
<code>[{pid, _}] = Registry.lookup(</code>	Crashes on no match; pid may be dead	5
<code>Enum.at(</code> inside a loop	Quadratic	6
<code>acc ++ [</code> in a reduce	Quadratic	6
<code>length(list) > 0</code>	Walks the whole list	6
<code>for i <- 1..n</code> where n can be 0	Counts down: runs twice	6
<code>String.length(</code> for byte limits	Graphemes are not bytes	7
<code>~r/.../</code> without <code>u</code> on user text	ASCII-only rules	7
Structs with password, token or key fields and no <code>@derive Inspect</code>	Secrets in logs	8
<code>use GenServer</code> without <code>@impl true</code>	Misspelled callbacks compile silently	8
<code>@fallback_to_any true</code>	Missing implementations become silent defaults	8
<code>Module.concat / String.to_atom</code> to pick modules	Untracked dependencies; atoms from input	8, 9
<code>defmacro</code> whose body a function could do	Unnecessary macro	9
<code>unquote(expr)</code> twice in one quote	Evaluates the argument twice	9
<code>System.get_env</code> in <code>config/*.exs</code> other than <code>runtime.exs</code>	Read on the build machine	10
<code>@attr Application.get_env(</code>	Frozen at compile time	10
<code>Mix.env()</code> in <code>lib/</code>	<code>Mix</code> doesn't exist in releases	10
<code>only: [:dev, :test]</code> on a dependency used in <code>lib/</code>	Missing from the release	10
<code>Application.put_env</code> in <code>async: true</code> tests	Shared global state	11
<code>refute_receive</code> right after an <code>async</code> action	Passes for late messages	11
<code>Process.sleep</code> in tests	Synchronising by waiting	11

Search for	Why it's suspicious	Ch.
@spec ... :: term()	Documents nothing	12
Log message built into a variable, then <code>Logger.debug(msg)</code>	Built even when debug is off	13
HTTP or <code>GenServer.call</code> in a telemetry handler	Runs synchronously in the emitter	13

2. Is the data what we think it is?

- ☐ Required fields are read with `map.key`, pattern matching or structs, not `map[:key]` (1)
- ☐ Guards that combine checks are safe for every input type, or split into separate clauses (1)
- ☐ Values compared in patterns are pinned with `^`; unused-variable warnings are errors in CI (1)
- ☐ Structs from external data go through `struct!/2` or a validating constructor (1)
- ☐ External data is validated at the boundary; specs are not relied on as checks (12)
- ☐ Environment strings are parsed to real types once, at startup (1, 10)
- ☐ Options are combined with `Keyword.merge/2` or checked with `Keyword.validate!/2` (1)
- ☐ Structs stay under 32 fields (1)

3. How are failures handled?

- ☐ Expected failures return `{:ok, _}` / `{:error, _}`; bang functions only where failure means a bug (2)
- ☐ Unexpected data fails fast or returns `{:error, reason}`, never a silent `nil` (1)
- ☐ No `try/rescue` for parsing, validation or routine branching (2)
- ☐ Every `with` step returns `:ok`, `{:ok, _}` or `{:error, _}`; `else` blocks are small and complete, or absent (2)
- ☐ `rescue` names specific exceptions (2)
- ☐ Options never change a function's return type (2)
- ☐ Resources that must be released are tied to a process's lifetime, not only to `after` (2)

4. Are processes used for the right reasons?

- ☐ Processes model state, concurrency or failure; no GenServers for code organisation (4)
- ☐ All interaction goes through client functions in the server's module (4)
- ☐ Callbacks never call the module's own client API (4)
- ☐ Calls return only the data the caller needs; closures capture only what they use (3, 4)
- ☐ No expensive functions run inside Agents; Agent updates are single calls (3)
- ☐ `Task.async` only where a task crash should crash the caller; otherwise `Task.Supervisor.async_nolink` (3)
- ☐ `async_nolink` results and `:DOWN` messages are handled, with `demonitor(ref, [:flush])` (3)

5. What happens under load?

- ❑ Concurrency over collections is bounded (`async_stream` with `max_concurrency`) (3)
- ❑ `async_stream` has an explicit timeout; `on_timeout: :kill_task` where partial results are fine (3)
- ❑ `init/1` returns quickly; slow setup is in `handle_continue/2` (4)
- ❑ Periodic work reschedules after it finishes (4)
- ❑ No `Enum.at/2` in loops, no `acc ++ [x]`, no `length/1` for emptiness (6)
- ❑ Large or slow sources use `Stream` and stop early; `Enum` is the default otherwise (6)
- ❑ Streams with side effects are consumed exactly once; every stream is consumed (3, 6)

6. What happens when something crashes?

- ❑ Children are ordered by dependency; Registries come before their users; `:rest_for_one` where needed (5)
- ❑ Every child in a supervisor has a unique `:id` (5)
- ❑ Per-connection and per-request processes are `restart: :temporary` (5)
- ❑ `DynamicSupervisor` restart limits suit the number of children, or `PartitionSupervisor` spreads them (5)
- ❑ Background work runs under a `Task.Supervisor` (3)
- ❑ Every custom `handle_info/2` ends with a catch-all clause (4)
- ❑ Processes with cleanup trap exits and have a suitable `:shutdown` (5)

7. Can processes be found reliably?

- ❑ Names are configurable; nothing assumes a single instance (4)
- ❑ Registry lookups handle `[]` and dead pids; requests go through via-tuples (5)
- ❑ Re-registering a key handles `{:error, {:already_registered, _}}` (5)

8. Is text handled correctly?

- ❑ Byte limits use `byte_size/1`; truncation never splits a character (7)
- ❑ Incoming text is normalised (NFC) before storage or comparison (7)
- ❑ Regexes on user text use the `u` modifier (7)
- ❑ Erlang charlists are converted with `List.to_string/1` at the boundary (7)
- ❑ Emptiness is tested with `== ""`; case conversion for keys uses an explicit mode (7)
- ❑ No code depends on map iteration order (6)
- ❑ Ranges that can be empty use an explicit step (`1..n//1`) (6)

9. Is polymorphism and metaprogramming justified?

- ❑ Protocols for data-driven polymorphism, behaviours for chosen implementations (8)
- ❑ Every callback is marked `@impl true` (8)
- ❑ Structs with secrets derive `Inspect` with `only:` or `except:` (8)
- ❑ No `Enum` on structs; no `@fallback_to_any` with business meaning (8)

- ❑ Multi-clause functions group one operation; implementations come from explicit maps (8)
- ❑ All `defimpls` live in compiled project code (8)
- ❑ Every macro needs compile-time behaviour or its arguments' code; otherwise it is a function (9)
- ❑ Macros bind arguments once, don't set caller variables, and don't read config outside `quote` (9)
- ❑ `use` injects only what must be injected and documents it; generated code is small (9)
- ❑ Module names are explicit; compile-time dependencies are checked with `mix xref` (9)

10. Will it work as a release?

- ❑ Everything environment-specific is read in `config/runtime.exs`; required values use `System.fetch_env!/1` (10)
- ❑ No `Application.get_env` in module attributes; `compile_env` where compile-time values are intended (10)
- ❑ No `Mix` calls in `lib/` (10)
- ❑ Dependencies used by `lib/` are available in all environments; `mix.lock` is committed (10)
- ❑ Libraries take configuration as options, not from the application environment (10)
- ❑ CI builds and boots the release (10)

11. Do the tests prove anything?

- ❑ `async: true` modules touch no global state (11)
- ❑ Test processes use `start_supervised!/1` and unique names (11)
- ❑ `assert_receive` timeouts fit the operation; `refute_receive` follows a synchronisation point (11)
- ❑ No `Process.sleep/1` to wait for work (11)
- ❑ Mocks are behaviour-based and allowed in the processes that use them (11)
- ❑ Doctests are deterministic and side-effect free (11)
- ❑ Expected values come from requirements, not the implementation (11)

12. Do the types tell the truth?

- ❑ Specs describe real inputs and outputs, including every error shape; no blanket `term()` (12)
- ❑ Compiler type warnings fail CI; a clean compile is not treated as proof (12)
- ❑ Dialyzer runs in CI with few, justified ignores; `@opaque` types are respected (12)

13. Can you see what it's doing?

- ❑ Log messages are built inside the `Logger` call; logs carry identifiers, not payloads (13)
- ❑ Request metadata is copied into tasks and spawned processes (13)
- ❑ Telemetry handlers are cheap, tolerant of unexpected metadata, and attached as `&Module.fun/4` (13)
- ❑ Durations and counts are metrics, not log lines; detach warnings are monitored (13)
- ❑ Production tracing is time-boxed and switched off explicitly (13)